

Cloud-Based Serverless Student Attendance Management System Using AWS

Project Submission Guide with Architecture, Deployment Steps, Flow Diagrams, and Source Code

Student: _____ **Department:** _____ **Year:** 4th Year

Duration: 4 weeks **Technology:** AWS Serverless, HTML, JavaScript, Python, DynamoDB

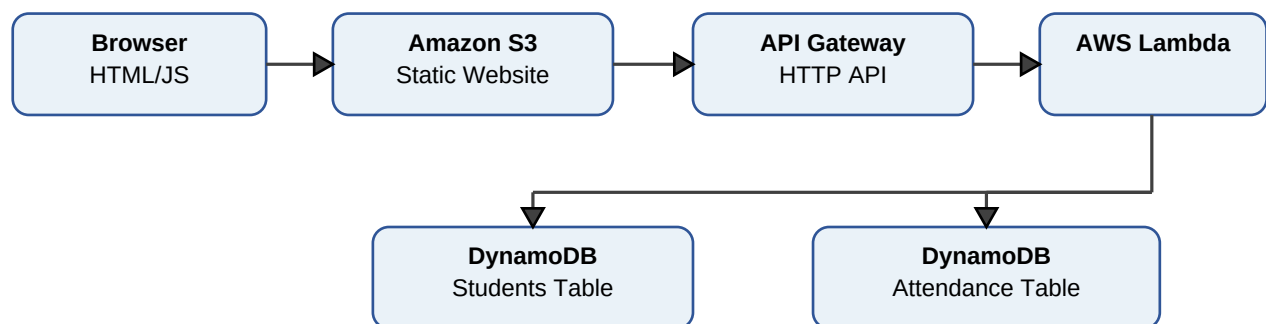
Abstract

This project is a cloud-based student attendance management system built using AWS serverless services. The application allows users to add student details, mark attendance, and view attendance records. The frontend is hosted as a static website on Amazon S3. Backend APIs are exposed through Amazon API Gateway and implemented using AWS Lambda functions written in Python. Student and attendance data are stored in Amazon DynamoDB.

Objectives

1. Build a simple cloud application without managing servers.
2. Store student and attendance records in DynamoDB.
3. Use API Gateway and Lambda to create REST-style APIs.
4. Host the frontend using Amazon S3 static website hosting.
5. Understand a practical serverless deployment flow.

System Architecture



Component	Purpose
Amazon S3	Hosts the static HTML, CSS, and JavaScript frontend.
API Gateway	Provides public API endpoints for add student, mark attendance, and view attendance.
AWS Lambda	Runs backend Python code without managing servers.
DynamoDB	Stores Students and Attendance records.

IAM Role	Allows Lambda to write and read DynamoDB data and write CloudWatch logs.
----------	--

Application Flow



The user enters data in the web page. JavaScript sends the data to API Gateway using fetch(). API Gateway invokes the correct Lambda function. Lambda validates and stores or reads data from DynamoDB. The response is returned to the web page and displayed to the user.

AWS Services Required

Service	Configuration
DynamoDB	Create two tables: Students and Attendance.
IAM	Create a Lambda role with DynamoDB and CloudWatch permissions.
Lambda	Create three functions: AddStudent, MarkAttendance, ViewAttendance.
API Gateway	Create HTTP API routes for the three Lambda functions.
S3	Create a bucket, upload index.html, and enable static website hosting.

Step-by-Step Deployment

Step 1 - Create DynamoDB Tables

Create the following tables in DynamoDB.

Table Name	Partition Key	Sort Key
Students	student_id (String)	None
Attendance	student_id (String)	date (String)

Step 2 - Create IAM Role for Lambda

Go to IAM -> Roles -> Create role. Select AWS service as trusted entity and Lambda as use case. Attach these policies: AmazonDynamoDBFullAccess and AWSLambdaBasicExecutionRole. Name the role StudentAttendanceLambdaRole.

Step 3 - Create Lambda Functions

Create three Lambda functions using Python 3.12 and attach the IAM role created above.

Step 4 - Create API Gateway Routes

Method	Route	Lambda Function
POST	/add-student	AddStudent
POST	/mark-attendance	MarkAttendance
GET	/view-attendance	ViewAttendance

Enable CORS for the API with Origin *, Headers *, and Methods GET, POST, OPTIONS. Deploy the API and copy the Invoke URL.

Step 5 - Update index.html

Replace the placeholder API URLs in index.html with your API Gateway Invoke URL and route names.

Step 6 - Host Frontend on S3

Create an S3 bucket, upload index.html, enable static website hosting, and set index.html as the index document. If public access is needed for demo, allow public read access using a bucket policy.

Step 7 - Test the Application

Open the S3 website endpoint. Add a student, mark attendance, and view attendance by entering the student ID. Verify the records in DynamoDB tables.

Source Code - Lambda Functions

AddStudent Lambda

```
import json
import boto3

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("Students")

def lambda_handler(event, context):
    body = json.loads(event["body"])

    student = {
        "student_id": body["student_id"],
        "name": body["name"],
        "department": body["department"],
        "year": body["year"]
    }

    table.put_item(Item=student)

    return {
        "statusCode": 200,
        "headers": {
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Headers": "*",
            "Access-Control-Allow-Methods": "*"
        },
        "body": json.dumps({"message": "Student added successfully"})
    }
```

MarkAttendance Lambda

```
import json
import boto3
from datetime import datetime

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("Attendance")

def lambda_handler(event, context):
    body = json.loads(event["body"])

    item = {
        "student_id": body["student_id"],
        "date": datetime.now().strftime("%Y-%m-%d"),
        "status": body["status"]
    }

    table.put_item(Item=item)

    return {
        "statusCode": 200,
        "headers": {
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Headers": "*",
            "Access-Control-Allow-Methods": "*"
        },
        "body": json.dumps({"message": "Attendance marked successfully"})
    }
```

ViewAttendance Lambda

```
import json
import boto3
from boto3.dynamodb.conditions import Key

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("Attendance")

def lambda_handler(event, context):
    student_id = event["queryStringParameters"]["student_id"]

    response = table.query(
        KeyConditionExpression=Key("student_id").eq(student_id)
    )
```

```
return {
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Headers": "*",
    "Access-Control-Allow-Methods": "*"
  },
  "body": json.dumps(response["Items"])
}
```

Frontend Code - index.html

Use this file as the frontend. Replace YOUR_API_ID and region with your actual API Gateway values.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Student Attendance Management System</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 0; background: #f4f4f4; }
    .container { max-width: 900px; margin: 30px auto; background: white; padding: 30px; border-radius: 10px; }
    h1 { text-align: center; }
    input, select { width: 100%; padding: 12px; margin: 10px 0; box-sizing: border-box; }
    button { padding: 12px 25px; cursor: pointer; }
    #result { background: #eeeeee; padding: 15px; margin-top: 20px; white-space: pre-wrap; }
  </style>
</head>
<body>
<div class="container">
  <h1>Student Attendance Management System</h1>

  <h2>Add Student</h2>
  <input id="student_id" placeholder="Student ID">
  <input id="name" placeholder="Student Name">
  <input id="department" placeholder="Department">
  <input id="year" placeholder="Year">
  <button onclick="addStudent()">Add Student</button>

  <hr>
  <h2>Mark Attendance</h2>
  <input id="att_student_id" placeholder="Student ID">
  <select id="status">
    <option value="Present">Present</option>
    <option value="Absent">Absent</option>
  </select>
  <button onclick="markAttendance()">Mark Attendance</button>

  <hr>
  <h2>View Attendance</h2>
  <input id="view_student_id" placeholder="Student ID">
  <button onclick="viewAttendance()">View Attendance</button>

  <div id="result"></div>
</div>

<script>
const baseUrl = "https://YOUR_API_ID.execute-api.us-east-1.amazonaws.com";
const addStudentURL = baseUrl + "/add-student";
const markAttendanceURL = baseUrl + "/mark-attendance";
const viewAttendanceURL = baseUrl + "/view-attendance";

async function addStudent() {
  const data = {
    student_id: document.getElementById("student_id").value,
    name: document.getElementById("name").value,
    department: document.getElementById("department").value,
    year: document.getElementById("year").value
  };
  const response = await fetch(addStudentURL, {
    method: "POST", headers: {"Content-Type": "application/json"}, body: JSON.stringify(data)
  });
  document.getElementById("result").innerText = await response.text();
}

async function markAttendance() {
  const data = {
    student_id: document.getElementById("att_student_id").value,
    status: document.getElementById("status").value
  };
  const response = await fetch(markAttendanceURL, {
    method: "POST", headers: {"Content-Type": "application/json"}, body: JSON.stringify(data)
  });
  document.getElementById("result").innerText = await response.text();
}

async function viewAttendance() {
  const studentId = document.getElementById("view_student_id").value;
  const response = await fetch(viewAttendanceURL + "?student_id=" + studentId);
  const result = await response.json();
}
```

```
        document.getElementById("result").innerText = JSON.stringify(result, null, 2);
    }
</script>
</body>
</html>
```

Testing Plan

Test Case	Input	Expected Result
Add student	ID: 12345, Name: Mohit, Department: CLOUD, Year: 2025	Student record is created in Students table.
Mark attendance	ID: 12345, Status: Present	Attendance record is created in Attendance table.
View attendance	ID: 12345	Attendance records are displayed on the web page.
Invalid route	Open POST route in browser using GET	API may show Not Found because browser uses GET.
CORS test	Click frontend button from S3 website	No CORS error should appear in browser console.

Troubleshooting Notes

1. {"message": "Not Found"}

This usually means the route or method is wrong. For example, opening /add-student in a browser sends GET, but the API route is POST. Test using the frontend, Postman, or API Gateway test option.

2. CORS error

Enable CORS in API Gateway and include Access-Control-Allow-Origin in Lambda responses.

3. DynamoDB access denied

Check the Lambda IAM role and confirm DynamoDB permissions are attached.

4. Data not saved

Check CloudWatch logs for Lambda errors and confirm table names are exactly Students and Attendance.

Four-Week Project Schedule

Week	Work Completed
Week 1	AWS account setup, DynamoDB tables, IAM role, Lambda basics.
Week 2	Created Lambda backend functions and API Gateway routes.
Week 3	Created frontend UI, integrated APIs, fixed CORS and route issues.
Week 4	Testing, documentation, screenshots, final report and project submission.

Conclusion

The project demonstrates a complete serverless application on AWS. It uses S3 for frontend hosting, API Gateway for API management, Lambda for backend processing, and DynamoDB for cloud database storage. The project is suitable for a final-year B.Tech submission because it shows cloud deployment, API integration, database design, and serverless architecture.